

Mémoire conversationnelle de Jev dans la Console — analyse et recommandation

30/09/2026 · à valider avant implémentation

1. Existant : pourquoi Jev perd le fil

Chaîne actuelle

Étape	Où	Ce qui circule
Saisie	Console Admin.dc.html (jSend → jevReply)	La conversation est affichée (état jm de l'écran), mais seul le dernier texte part au serveur.
Appel	admin-api.js → POST /api/admin/assistant/messages	{ context: { section }, text } : ni historique, ni identifiant de conversation.
Traitement	ConsoleController.jev → JevSqlService.ask	1 appel pour produire une requête SQL (ou une réponse directe), 1 correction au plus, 1 appel pour rédiger la réponse.
Modèle	LlmService.complete → LlmClient.generate	messages: [{ role: 'user', content: prompt }] : un seul message , sans aucun tour précédent.

Cause : les API des modèles sont sans état, et Jev ne leur renvoie jamais les échanges précédents. Le serveur ne conserve rien. L'écran garde bien l'historique, mais il ne l'envoie pas, et le perd au rechargement de la page. À la question « Quel est le périmètre fonctionnel du projet ? », le modèle ne voit donc que cette phrase.

Deuxième cause, propre à l'exemple RISE : le dictionnaire des données de la Console (schéma jev , 32 vues) ne contient pas l'objet « Info projet ». Seul celui du Cockpit a la vue infos_projet . Même avec une mémoire, Jev saurait de quel projet on parle, mais pas où lire son périmètre fonctionnel.

Mesures actuelles (fonction guidage , 7 derniers jours, 41 appels) :

- modèle principal : Claude Haiku 4.5 (Anthropic), secours OpenAI ;
- en moyenne 11 600 jetons d'entrée (19 700 au plus) et 190 jetons de sortie par appel ;
- 2 à 3 appels par question.

Le prompt système (base, Identité, Soul, skill « Guidage console », page, dictionnaire) représente presque toute l'entrée. Il **n'est pas mis en cache**, et il ne pourrait pas l'être en l'état : il contient l'heure courante (nowParis()) et la page ouverte, qui changent d'un appel à l'autre.

2. Ce que proposent les fournisseurs

Documentations officielles consultées le 30/09/2026.

Fournisseur	Géré par le fournisseur	Reste à notre charge	Limites
Anthropic (Jev aujourd'hui)	Rien pour l'état : l'API Messages est sans état. Cache de prompt explicite (<code>cache_control</code> , 5 min ou 1 h, lecture à 0,1×). Compaction côté serveur (bêta, pas sur Haiku 4.5). Outil mémoire <code>memory_20250818</code> : le modèle écrit des notes, mais le stockage est à fournir par l'application . Memory stores des Managed Agents (bêta, agents hébergés : hors de notre architecture).	Historique, résumé, mémoire long terme, droits.	Contexte de 200 K (Haiku 4.5). Cache à partir de 4 096 jetons sur Haiku 4.5.
OpenAI	Responses API (<code>previous_response_id</code> , réponses gardées 30 jours) et Conversations API (conservées jusqu'à suppression). Compaction côté serveur. Cache automatique (1 024 jetons au moins, 0,1×).	Mémoire long terme (aucun outil dédié).	L'historique stocké est refacturé en entrée à chaque tour. Incompatible avec la ZDR et la résidence UE. Assistants API retirée le 26/08/2026.
Google Gemini	Interactions API (<code>previous_interaction_id</code> , 55 jours en offre payante). Cache implicite (explicite seulement sur l'API legacy ou Vertex AI).	Le chat du SDK renvoie tout l'historique depuis le client. Mémoire long terme.	Refacturation de l'historique non documentée. ZDR impossible avec l'état stocké.
Mistral	Conversations API (bêta, conservées jusqu'à suppression, hébergement UE). Cache via <code>prompt_cache_key</code> (0,1×, non garanti).	Mémoire long terme.	Conversations exclues de la ZDR. Refacturation et durée du cache non documentées.

À retenir :

- Aucun fournisseur ne nous épargne le coût de l'historique : il est refacturé, ou pas documenté.
- Aucun état n'est **portable** d'un fournisseur à l'autre. Or Jev bascule d'Anthropic vers OpenAI en cas d'indisponibilité : un état stocké chez l'un serait perdu chez l'autre.
- Le seul levier natif à retenir est le **cache de prompt**. Il est automatique chez OpenAI et Gemini ; chez Anthropic, il se déclenche par un marqueur à ajouter.

3. Approches possibles

#	Approche	Compréhension du contexte	Mise en œuvre	Coût (jetons, infra)	Latence	Multi-fournisseur	Entre sessions	Multi-utilisateur	Contrôle utilisateur
A	État natif du fournisseur	Bonne	Moyenne, une par fournisseur	Historique refacturé	=	Non (perdu au secours)	Oui, chez le fournisseur	À gérer	Faible
B	Historique renvoyé, fenêtre glissante	Bonne sur les N derniers tours	Faible	+2 à 4 K jetons par appel	≈ =	Oui	Oui si stocké chez nous	Oui	Oui

#	Approche	Compréhension du contexte	Mise en œuvre	Coût (jetons, infra)	Latence	Multi-fournisseur	Entre sessions	Multi-utilisateur	Contrôle utilisateur
C	B + résumé des tours sortis de la fenêtre	Bonne, même en conversation longue	Faible à moyenne	Borné (+1 petit appel tous les N tours)	= (résumé après la réponse)	Oui	Oui	Oui	Oui
D	Contexte actif suivi (projet, entité)	Bonne pour les entités, faible pour le reste	Moyenne (détection fiable délicate)	Très faible	=	Oui	Oui	Oui	Oui
E	Reformulation de la question	Très bonne pour la requête SQL	Faible	+1 petit appel	+0,5 à 1 s	Oui	—	—	—
F	Mémoire long terme en SQL (faits par utilisateur)	Bonne pour les préférences et les projets suivis	Moyenne	Faible (≈ 1 K jetons injectés)	=	Oui	Oui	Oui	Oui (voir, effacer)
G	Mémoire vectorielle ou frameworks (Mem0, Zep, LangGraph)	Bonne à grande échelle	Élevée (service en plus, Python)	Infra en plus	+	Oui	Oui	Oui	Variable

Cache du prompt système (transverse) : figer le prompt (sortir l'heure et la page du bloc mis en cache) et le marquer pour Anthropic. Les ~11 K jetons fixes sont alors facturés à 0,1× dans une conversation active. Cette économie compense largement le coût de l'historique.

4. Recommandation

Retenu : B + C + cache, stockés chez nous ; F ensuite ; E seulement si les tests le demandent.

Pourquoi :

- C'est la seule approche qui fonctionne à l'identique quand Jev bascule d'Anthropic vers son secours OpenAI.
- La mémoire reste dans notre base, par administrateur : on peut la voir, l'effacer, la conserver une durée limitée, et elle respecte le RGPD.
- Le coût final est **inférieur à l'actuel** grâce au cache.
- Elle suffit pour le Text-to-SQL : le modèle qui écrit la requête voit les tours précédents et résout « le projet » tout seul.
- La reformulation (E) n'ajouterait qu'un appel. Elle reste en réserve si les tests montrent des requêtes SQL mal ciblées.
- Les frameworks (G) sont surdimensionnés pour une trentaine d'administrateurs.

Étape 1 : garder le fil (règle le cas RISE)

Stockage

- Tables `jev_conversations` (compte, date, résumé) et `jev_messages` (rôle, texte, vues consultées, requête SQL, date).
- Chaque conversation est propre à un administrateur. Elle est supprimée au bout de 30 jours (`JEV_CONVERSATION_RETENTION_DAYS`).

API

- `POST /api/admin/assistant/messages` accepte `conversationId` et le renvoie. Sans identifiant, une nouvelle conversation est créée.
- Nouvelles routes : `GET /api/admin/assistant/conversations/current` (reprise après rechargement) et `POST .../conversations` (« Nouvelle conversation »).

Envoi au modèle

- `LlmClient.generate` accepte une liste de messages (Anthropic, OpenAI, Gemini).
- `JevSqlService.ask` envoie les **10 derniers échanges** (question et réponse rédigée, sans les lignes SQL), bornés à environ 4 000 jetons (`JEV_HISTORY_TURNS` , `JEV_HISTORY_MAX_TOKENS`). Ils sont joints à l'appel qui écrit la requête comme à celui qui rédige la réponse.

Résumé

- Les tours sortis de la fenêtre sont résumés en quelques lignes (projet ou objet en cours, décisions, faits utiles).
- Le résumé est mis à jour après la réponse, par un petit appel, sans ralentir Jev.

Cache

- Le prompt système est figé : l'heure et la page ouverte passent dans un bloc séparé, après le dictionnaire.
- Il porte le marqueur de cache Anthropic.

Dictionnaire de la Console : ajout de la vue `infos_projet` (mêmes rubriques que dans le Cockpit), pour que la réponse de l'exemple RISE existe.

Écran de la Console

- En-tête du panneau Jev : bouton « Nouvelle conversation » (icône et info-bulle).
- La conversation reprend après un rechargement de la page.
- Changer de page ne coupe pas le fil.

Étape 2 : mémoire long terme (à évaluer après l'étape 1)

- **Faits mémorisés** : des faits courts par administrateur (« suit le projet RISE », « préfère les montants en k€ »), proposés par le modèle en fin de conversation. Ils sont stockés en SQL et injectés en tête de conversation (environ 1 K jetons au plus).
- **Contrôle** : écran « Mon profil › Mémoire de Jev » pour voir, supprimer, tout effacer ou désactiver.
- **Consentement** : aucun fait n'est mémorisé à l'insu de l'administrateur. Chaque ajout est signalé dans la conversation et peut être annulé.

5. Scénarios de test

Les tests automatiques utilisent un modèle simulé qui enregistre les messages reçus. Les tests manuels se font avec Claude Haiku 4.5.

#	Scénario	Attendu
1	RISE : « Donne-moi des informations sur le projet RISE », puis « Quel est le périmètre fonctionnel du projet ? »	Le 2e appel contient le 1er échange. La réponse cite le périmètre de RISE sans demander de quel projet il s'agit.
2	Changement de sujet : RISE, puis « Qui est administrateur de la plateforme ? », puis « Depuis quand ? »	« Depuis quand » porte sur les administrateurs, pas sur RISE.
3	Retour au sujet : après le scénario 2, « Et pour le projet, combien de chantiers ? »	Répond pour RISE.
4	Conversation très longue : 40 échanges	Entrée bornée (10 tours et le résumé). Le projet évoqué au 1er tour reste compris au 40e, grâce au résumé.
5	Nouvelle conversation	« Quel est son périmètre ? » ne suppose plus RISE, et Jev demande de quel projet on parle.
6	Rechargement de la page	La conversation reprend, avec le même fil.
7	Deux administrateurs	Aucun ne voit ni n'influence la conversation de l'autre (conversation d'un autre compte : 404).
8	Secours fournisseur : Anthropic indisponible en cours de conversation	Le modèle OpenAI reçoit le même historique et garde le fil.
9	Cache : 2 questions à moins de 5 minutes d'intervalle	<code>cache_read_input_tokens</code> > 0 au 2e appel. Coût d'entrée ≤ coût actuel.
10	Conservation : conversation de plus de 30 jours	Supprimée par la tâche de purge.